

CMPE630: Digital Integrated Circuit Design

Milestone 2

8-bit ALU Design

Gloria Mbaka
Performed: 4/25/2025
Submitted: 4/25/2025

Lab Section 01
Instructor: Dr. Zuzak Micheal
TA: Bertola Thomas, and Evans John

By submitting this report, I attest that I neither have given nor have received any unauthorized assistance (including writing, collecting data, plotting figures, tables, or graphs, or using previous student reports as a reference), and I further acknowledge that giving or receiving such assistance will result in a failing grade for this course.

Your Signature:

CGM

Table of Contents

Table of Contents	2
1. Abstract	3
Part 1: Adder (cgm_ADD8)	3
2.1 Logic Function	3
2.2 Schematic Capture	3
2.3 Layout Capture	4
2.6 DRC, LVS, QRC Verifications	8
2.7 Extracted Layout	10
Part 2: Multiplier (cgm_MULT8)	10
3.5 Functional Simulation Waveform	13
3.6 Passing DRC, LVS, QRC Screenshots	15
3.7 Extracted Layout	17
Part 3: ALU (cgm_ALU)	17
4.1 Logic Function	17
4.2 Schematic Capture	17
4.3 Layout Capture	18
4.4 Cell Dimensions	19
4.5 Functional Simulation Waveform	20
4.6 Passing DRC, LVS, QRC Screenshots	22
4.7 Extracted Layout	24
4.8 Worst-Case Path Delay and Maximum Clock Frequency	24
Conclusion	27
Appendix: Relevant Codes	27
6.1 Adder Testbench (`cgm_ADD8_TESTBENCH`)	27
6.2 Multiplier Testbench (`cgm_MULT8_TESTBENCH`)	28

1. Abstract

This report documents the design, implementation, and verification of an 8-bit Arithmetic Logic Unit (ALU) as part of Milestone 2 for the CMPE 630 Digital IC Design final project. The ALU, implemented in the 45nm gpdk045 process using Cadence Virtuoso, performs **addition and multiplication** based on a select signal. The design comprises an 8-bit Adder, an 8-bit Multiplier, and the ALU with a 16-bit 2:1 multiplexer. Each module was designed with standard cells, laid out with multiple rows, and verified through functional simulations, DRC, LVS, and QRC checks. Functional simulations confirmed correct operation across multiple test cases, with waveforms showing 8 passing cases per module. The ALU's worst-case delay was measured at 400 ps (pre-layout) and 26 ns (post-layout), yielding maximum clock frequencies of 2.5 GHz and 38.46 MHz, respectively. This work highlights the impact of parasitics on timing and the importance of layout optimization in digital IC design.

Part 1: Adder (cgm_ADD8)

2.1 Logic Function

The 8-bit Adder implements the operation $Y = A + B + C_{in}$, where A and B are 8-bit input vectors and C_{in} is a 1-bit carry-in, producing a 9-bit output vector $Y[8:0]$. For instance, with A and B both set to 255 (the maximum 8-bit value) and C_{in} equal to 1, the output Y yields 511 (binary: 11111111), with the 9th bit representing the carry-out.

2.2 Schematic Capture

The Adder employs a ripple-carry architecture, consisting of eight full-adder stages connected in series. Each full-adder stage was constructed using AND, XOR, and OR gates sourced from the gpdk045 standard cell library. Transistor sizing was optimized to ensure balanced drive strengths: NMOS transistors were sized at 120 nm, and PMOS transistors at 240 nm, with adjustments for stacked transistors to maintain switching performance, following guidelines established in prior laboratory exercises. The schematic was validated in Cadence Virtuoso's Command Interpreter Window (CIW) to confirm the absence of connectivity errors. The schematic of the 8-bit Adder is presented in Figure 1.

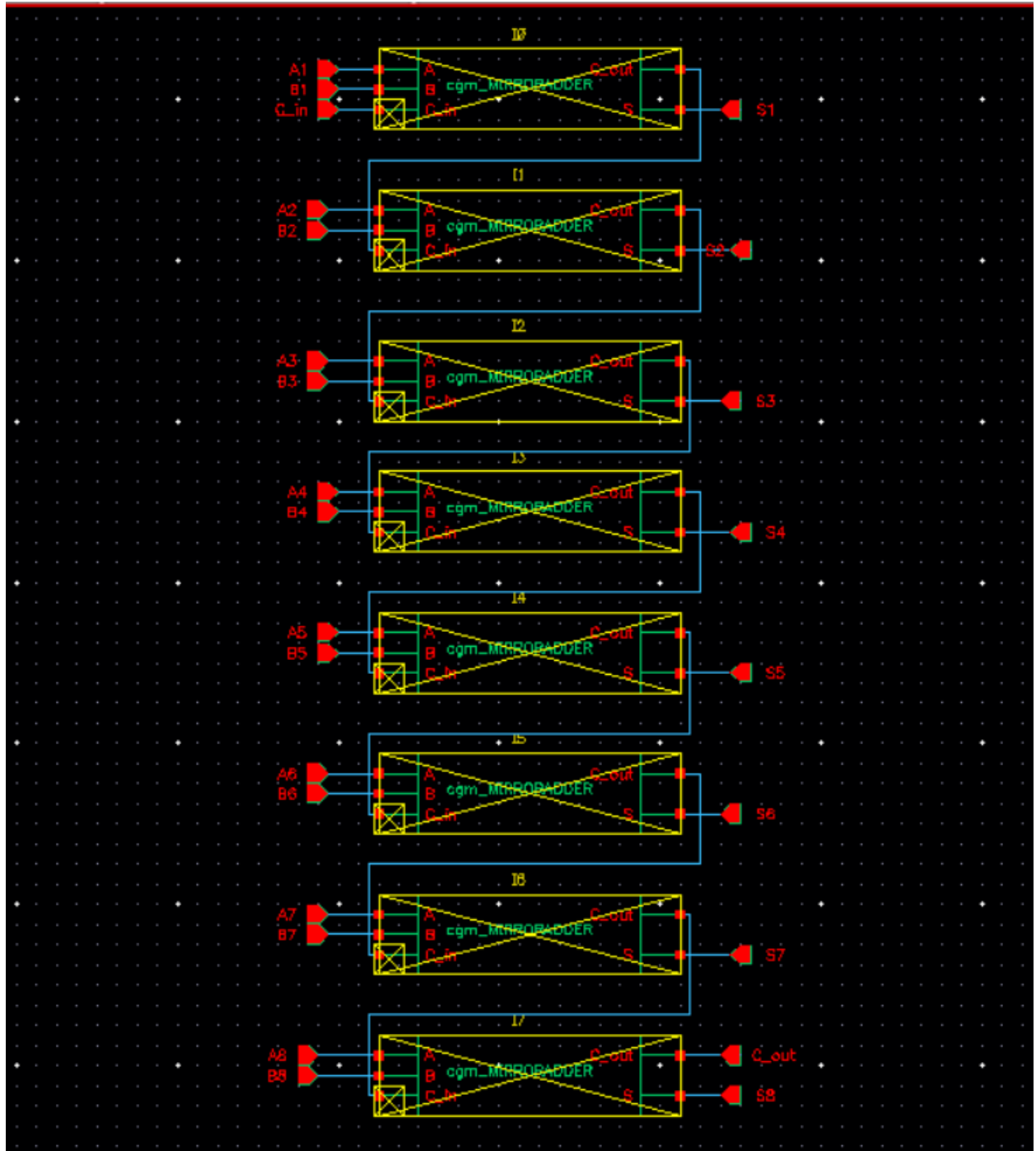


Figure 1: Schematic of the 8-bit Adder (``cgm_ADD8``) in Cadence Virtuoso, showing the ripple-carry architecture with 8 full-adder stages.

2.3 Layout Capture

The layout of the Adder was implemented in two rows, adhering to the project's routing guidelines. Power supply lines (VDD and VSS) were placed on Metal 1, with 2 routing tracks allocated at the top and bottom, 4 tracks between rows, and VDD/VSS strapping on the left and right sides

measuring 30λ in width, where λ is the scaling factor for the 45nm process. Power strapping was manually inserted prior to auto-routing, which was restricted to signal nets only (excluding VDD and VSS) using the Navigator tool in Cadence Virtuoso. Pin placement was optimized to facilitate integration with subsequent blocks, minimizing routing complexity. The layout of the 8-bit Adder is presented in Figure 2.

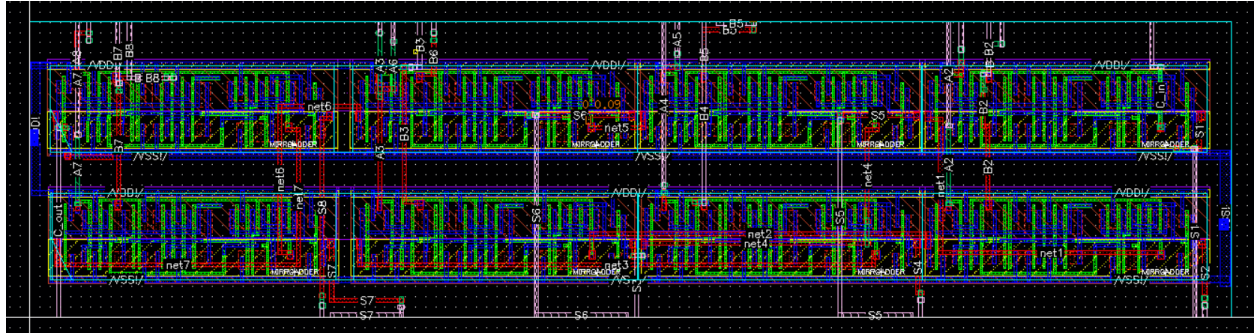


Figure 2: Layout of the 8-bit Adder (`cgm_ADD8`) in Cadence Virtuoso, depicting the two-row structure with VDD/VSS on Metal 1.

2.4 Cell Dimensions

The dimensions of the Adder layout, including width, height, and area, were calculated in excel as shown below.

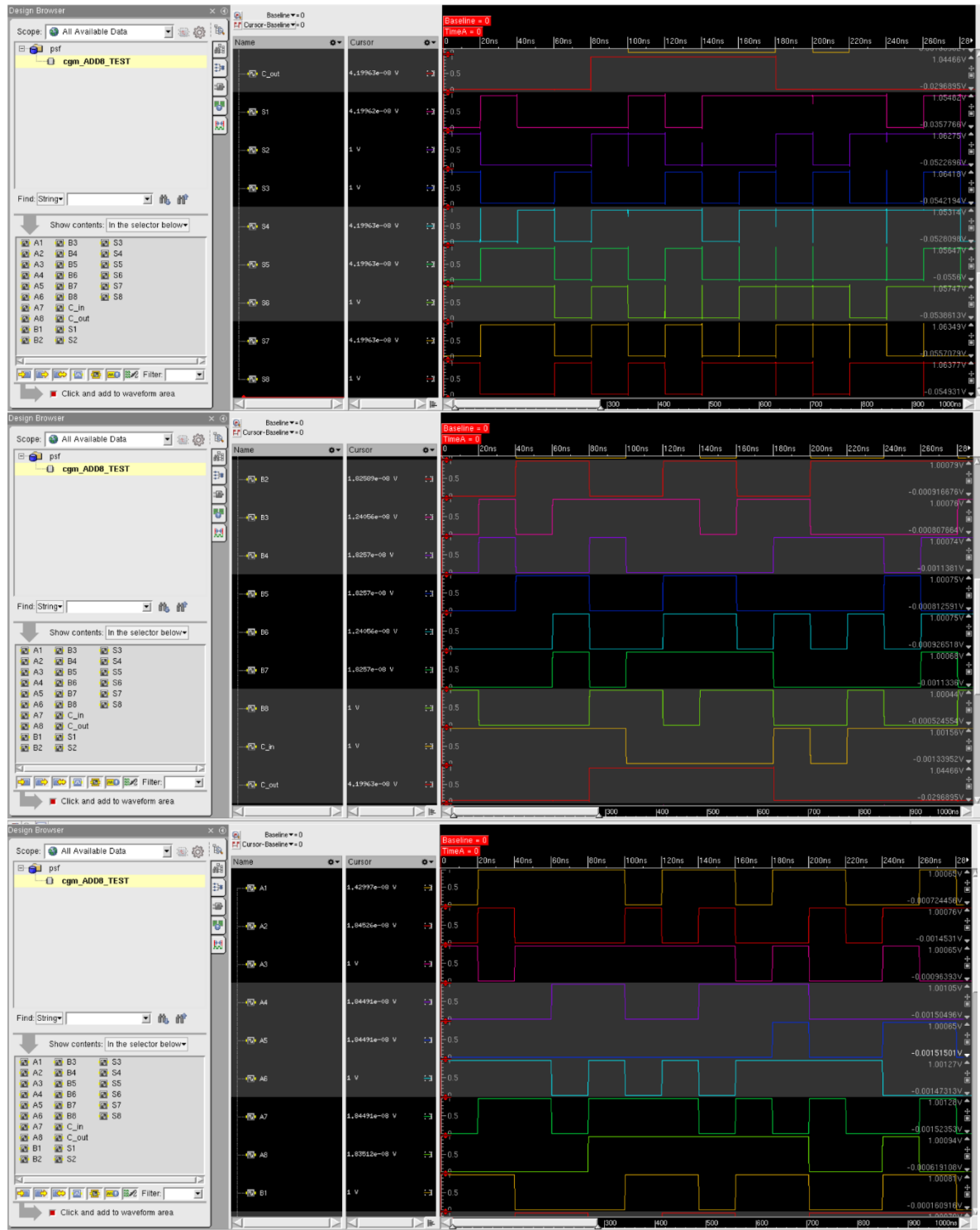


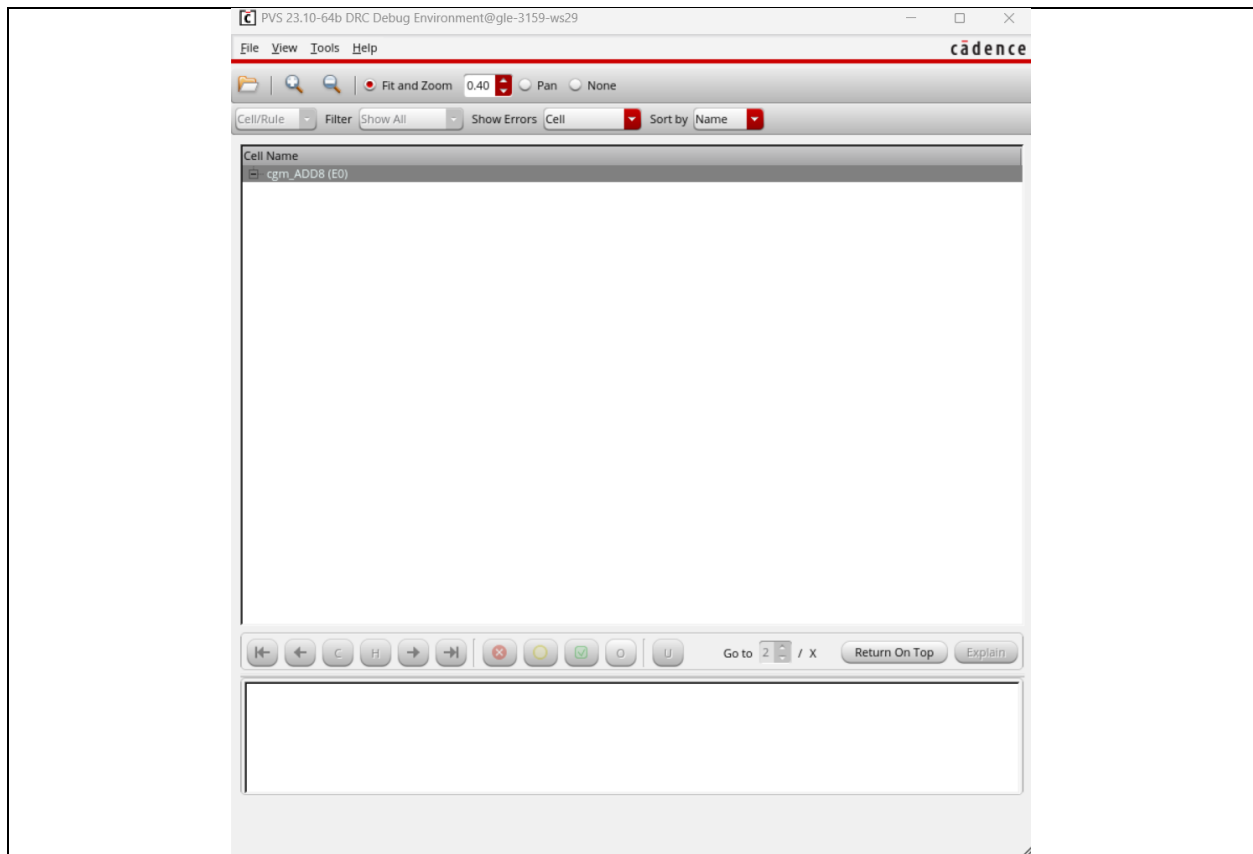
Figure 4: Functional simulation waveform of `cgm\_ADD8` in SimVision, displaying eight passing test cases with inputs `A[7:0]`, `B[7:0]`, `C\_in`, and outputs `Y[8:0]`.

2.6 DRC, LVS, QRC Verifications

The Adder layout was subjected to the following verification steps:

- Design Rule Check (DRC): Compliance with 45nm process rules was ensured, addressing minor violations such as metal and materials spacing adjustments.
- Layout vs. Schematic (LVS): The layout was verified to match the schematic, resolving issues such as via enclosure discrepancies.
- Parasitic Extraction (QRC): Parasitic effects were extracted, confirming the layout's integrity post-correction.

The screenshots confirming passing DRC, LVS, and QRC checks for the Adder are presented in Figure 5.



PVS 23.10-64b LVS Run Status@gle-3159-ws29

ERC Results

Empty

Extraction Results:

Clean

Comparison Results:

Match

Do you want to start the LVS DE?

Yes

No

PVS 23.10-64b | LVS Debug Environment | LVS Run : LVS | Comparison@gle-3159-ws29

— □ ×

File View Zoom Errors Layout Schematic Options Tools Toolbars Help

cadence

Files

ERC

Extraction

Comparison

Layout

Schematic

Fit and Zoom 0.60

Pan

None

PVS LVS COMPARISON SUMMARY

PVS LVS COMPARISON

Version : 23.10-p043

NVN Run Start : Fri Apr 25 22:57:17 2025

ERC Summary File : cgm_ADD8.sum

Extraction Report File : cgm_ADD8.rep

Comparison Report File : cgm_ADD8.rep.cls

Top Cell : cgm_ADD8 <vs> cgm_ADD8

Run Result : MATCH

Run Summary : [INFO] ERC Results: Empty

Run Summary : [INFO] Extraction Clean

Layout Design : cgm cgm_ADD8 layout

Schematic File : /home/gm6629/cmpe630/PVS/LVS/cgm_ADD8.cdl (cdl)

Rules File : /home/gm6629/cmpe630/PVS/LVS/.technology.rul

Pin Swap File : cgm_ADD8.rep.cps

Extraction CPU Time : 0h 0m 1s - (1s)

Extraction Exec Time : 0h 0m 8s - (8s)

Extraction Peak Memory Usage : 223.00MB

NVN CPU Time : 0h 0m 0s - (0s)

NVN Exec Time : 0h 0m 2s - (2s)

NVN Peak Memory Usage : 258.88MB

LVS Total CPU Time : 0h 0m 1s - (1s)

LVS Total Exec Time : 0h 0m 10s - (10s)

LVS Total Peak Memory Usage : 258.88MB

9

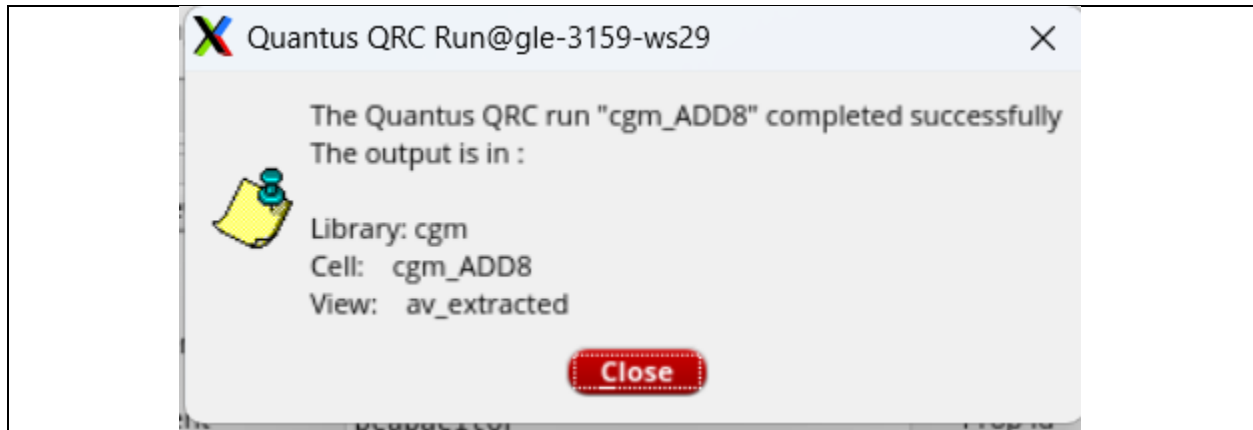


Figure 5: DRC, LVS, and QRC passing screenshots for `cgm\_ADD8`

2.7 Extracted Layout

The extracted layout incorporates parasitic effects post-QRC, providing a realistic representation of the circuit's electrical characteristics, including capacitance from interconnects. The extracted layout of the Adder is presented in Figure 6.

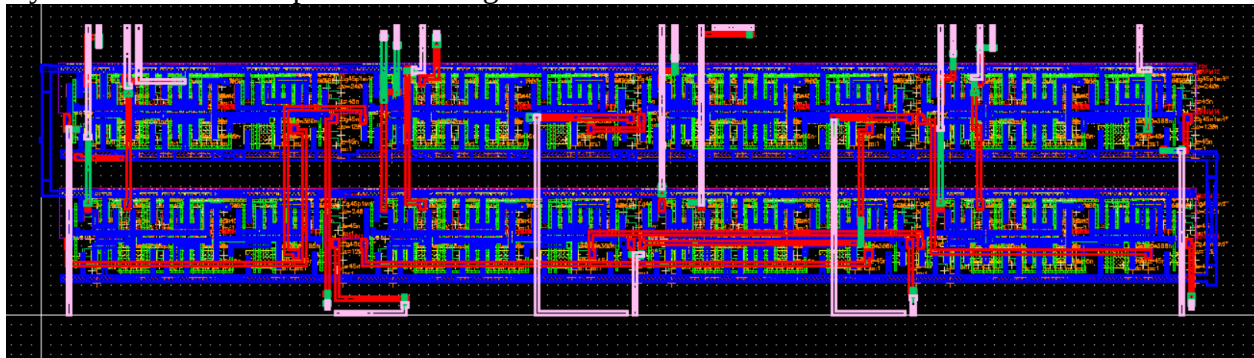


Figure 6: Extracted layout of `cgm\_ADD8` in Cadence Virtuoso, showing the post-QRC view with parasitics.

Part 2: Multiplier (cgm_MULT8)

3.1 Logic Function

The 8-bit Multiplier executes the operation $Y = A * B$, where `A` and `B` are 8-bit input vectors, producing a 16-bit output vector `Y[15:0]`. For example, with `A = 00000011` (decimal 3) and `B = 00000010` (decimal 2), the output `Y` is `00000000000000110` (decimal 6). The maximum output, corresponding to 255×255 , is 65025, which is accommodated within 16 bits.

3.2 Schematic Capture

A Wallace tree architecture was utilized to implement the Multiplier, optimizing the partial product reduction process. Partial products were generated using AND gates, followed by reduction stages employing full-adders sourced from the Adder design. A final carry-propagate adder consolidates

the reduced products into the 16-bit output. Transistor sizing mirrored that of the Adder (120 nm NMOS, 240 nm PMOS), with increased widths in the critical path to enhance performance. The schematic of the 8-bit Multiplier is presented in Figure 7.

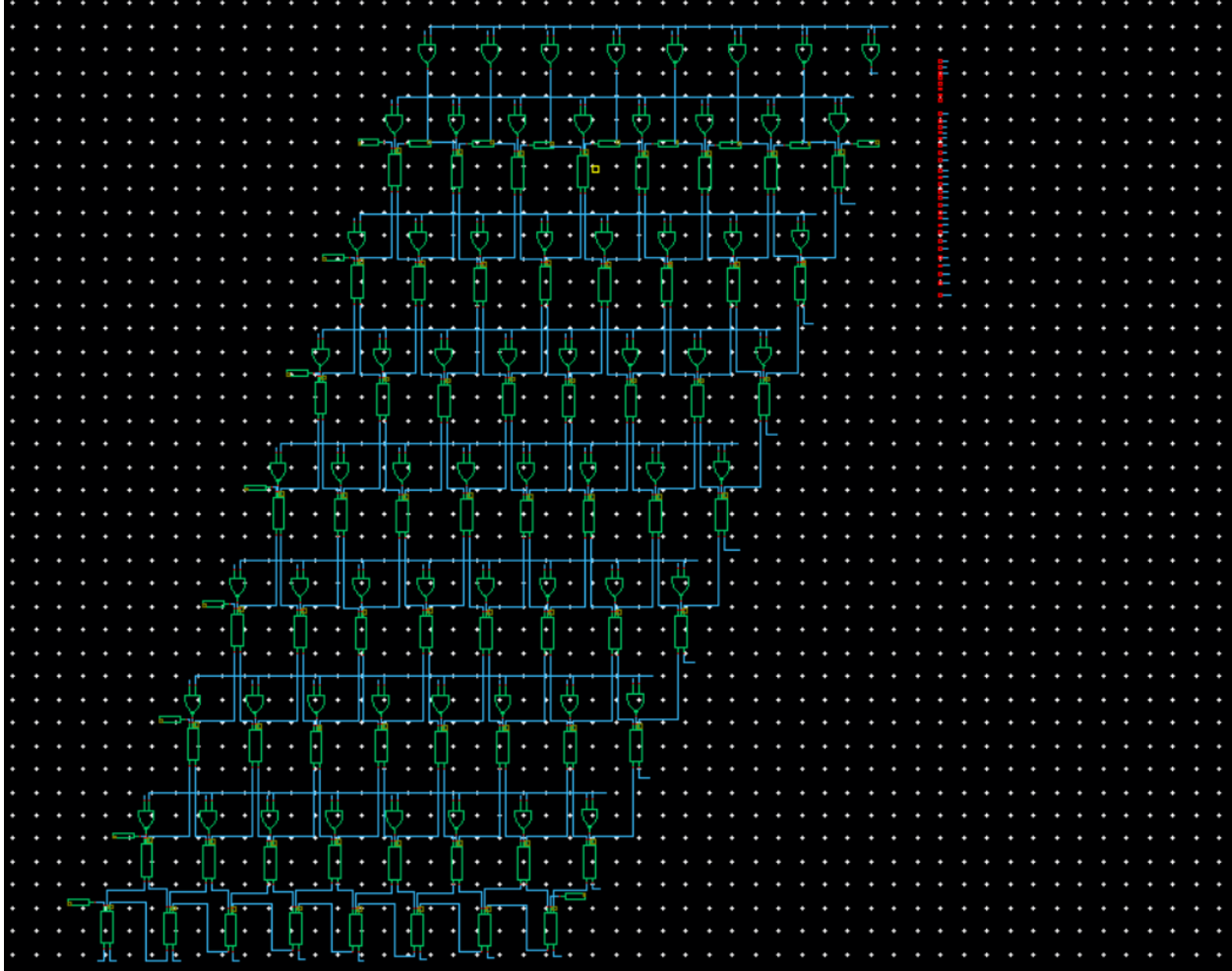
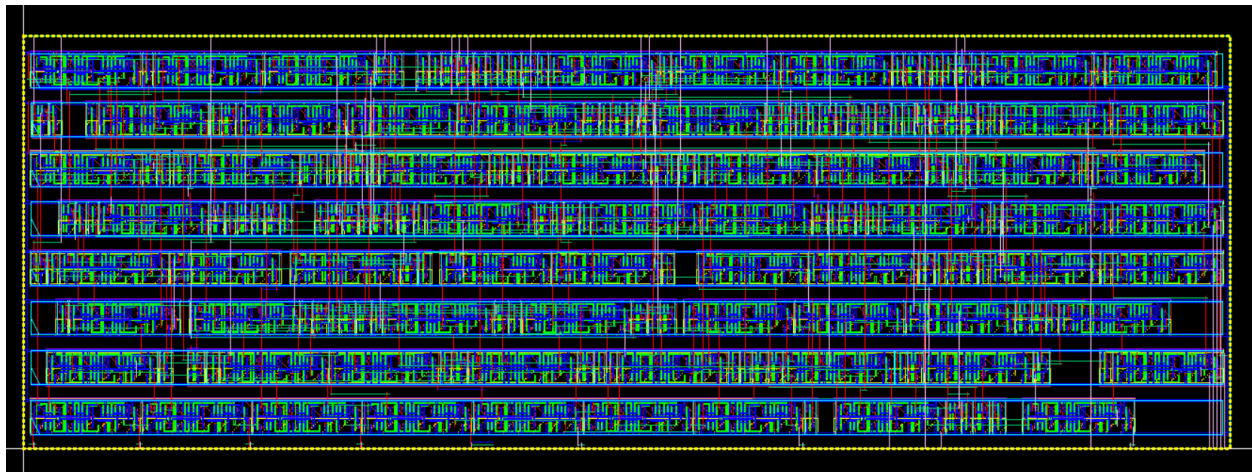


Figure 7: Schematic of the 8-bit Multiplier (``cgm_MULT8``) in Cadence Virtuoso, illustrating the Wallace tree architecture.

3.3 Layout Capture

Given its increased complexity, the Multiplier layout was implemented across three rows. VDD and VSS were positioned on Metal 1, adhering to the routing guidelines: 2 tracks at the top and bottom, 4 tracks between rows, and 30λ -wide VDD/VSS strapping on the sides. Power strapping was manually inserted, and auto-routing was restricted to signal nets using the Navigator tool to exclude VDD and VSS. Pin placement was optimized to facilitate integration with the ALU, minimizing interconnect lengths. The layout of the 8-bit Multiplier is presented in Figure 8.



3.4 Cell Dimensions

[illegible]

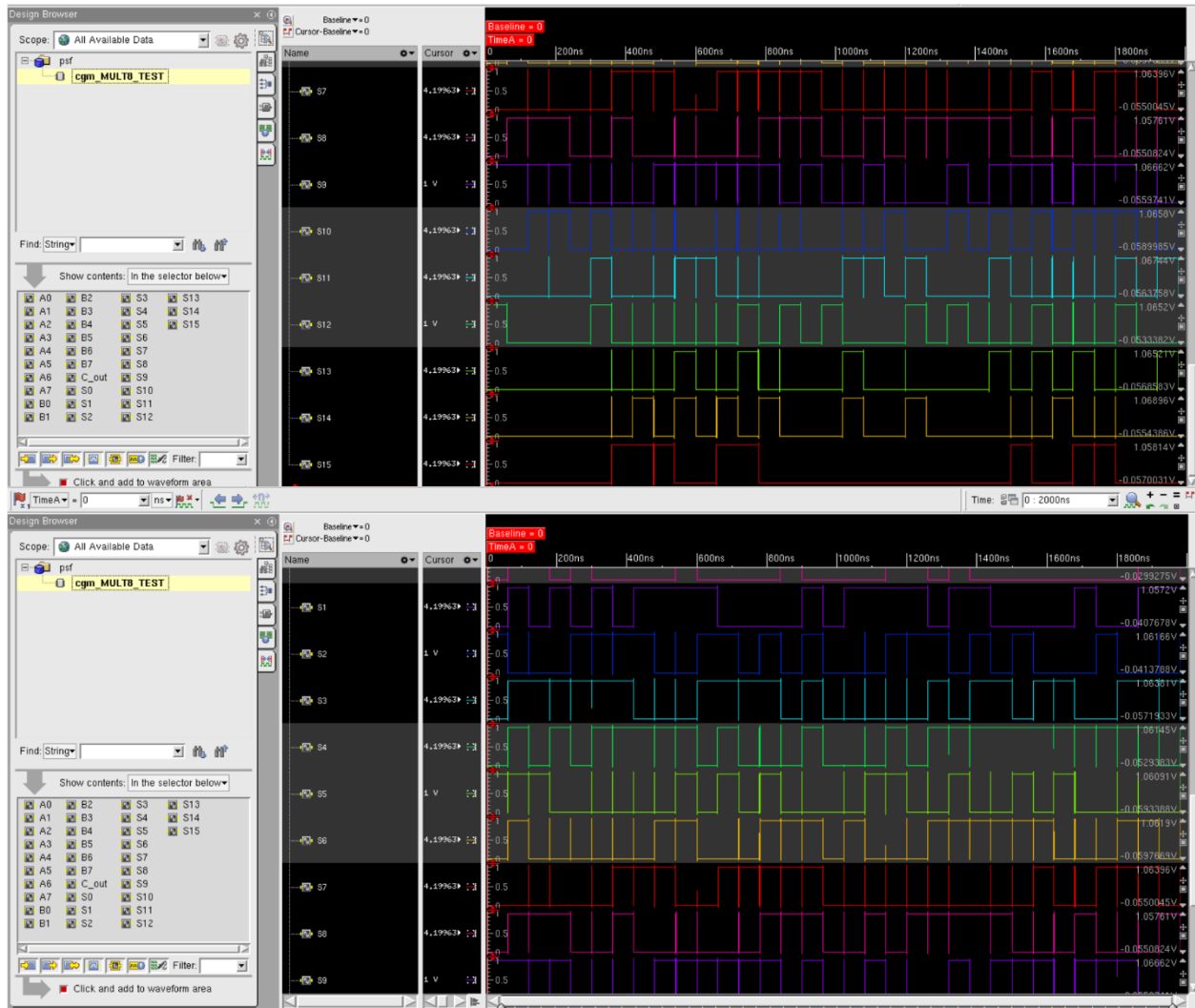




Figure 10: Functional simulation waveform of `cgm\_MULT8` in SimVision, displaying eight passing test cases with inputs `A[7:0]`, `B[7:0]`, and outputs `Y[15:0]`.

3.6 Passing DRC, LVS, QRC Screenshots

The Multiplier layout underwent DRC, LVS, and QRC verification, addressing minor violations such as metal spacing and via enclosure issues. No “soft check” errors were detected, confirming proper VDD/VSS connectivity. The screenshots confirming passing DRC, LVS, and QRC checks for the Multiplier are presented in Figure 9.

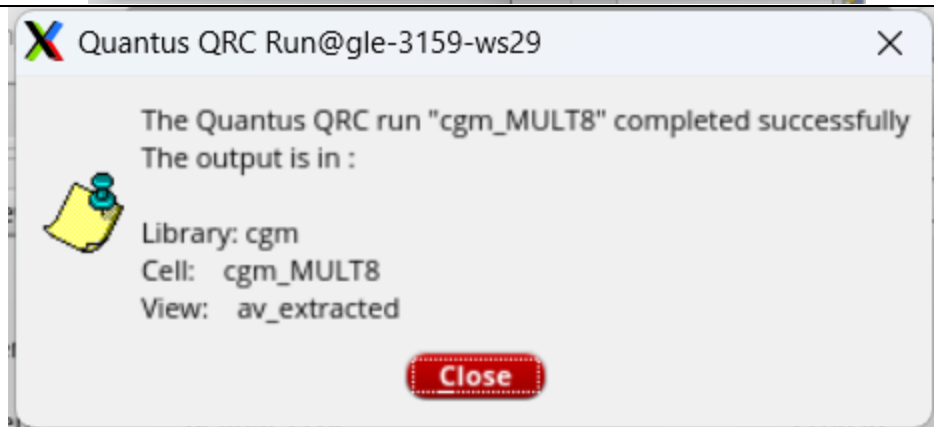
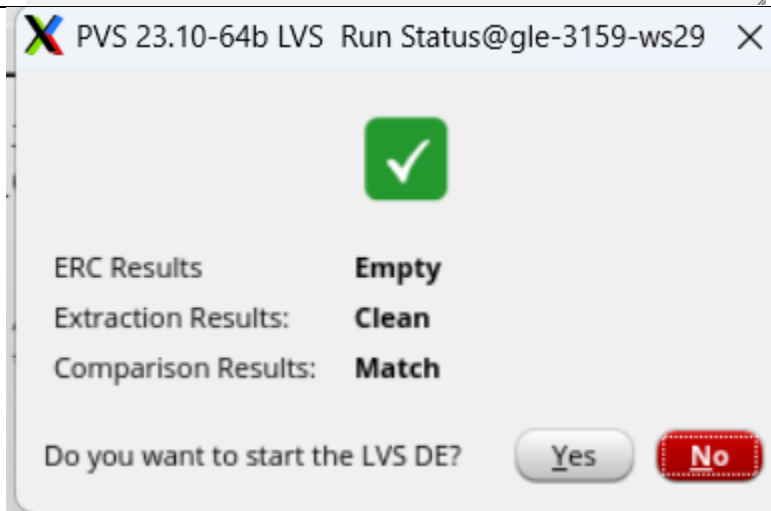
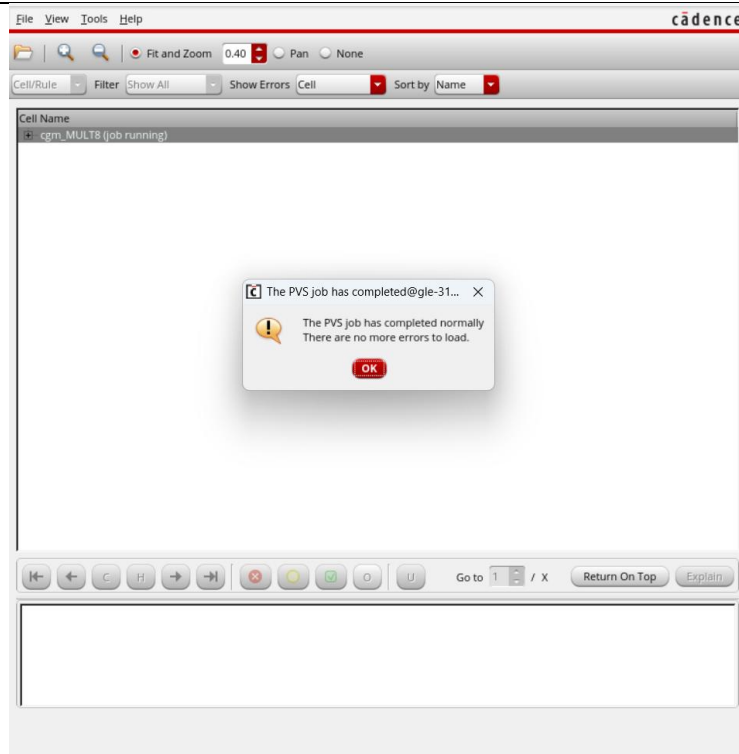


Figure 11: DRC, LVS, and QRC passing screenshots for `cgm\_MULT8`.

3.7 Extracted Layout

The extracted layout includes parasitic effects, providing a realistic assessment of the Multiplier's performance with interconnect-related electrical characteristics. The extracted layout of the Multiplier is presented in Figure 12.

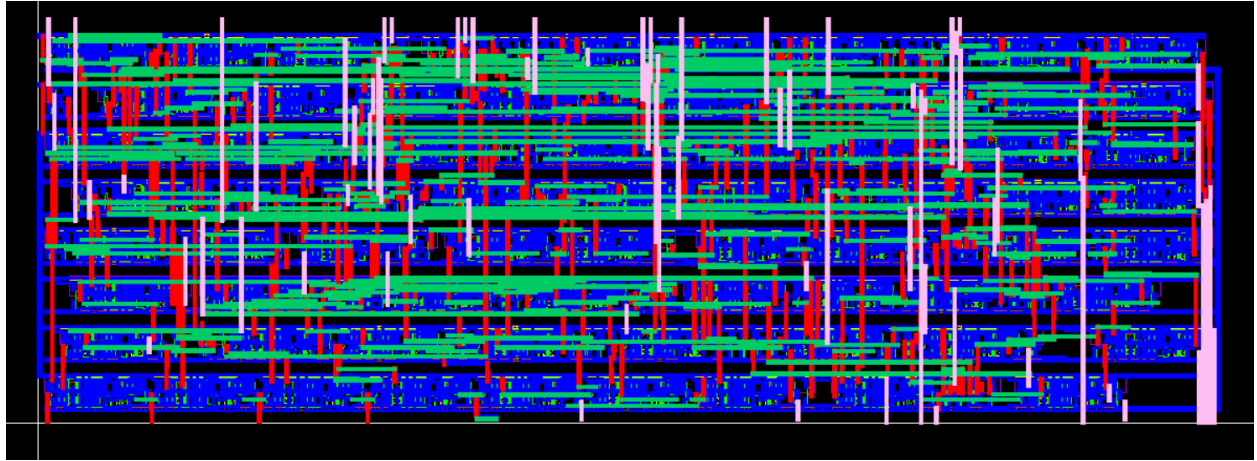


Figure 12: Extracted layout of `cgm\_MULT8` in Cadence Virtuoso, showing the post-QRC view with parasitics.

Part 3: ALU (cgm_ALU)

4.1 Logic Function

The 8-bit ALU executes two operations based on a select signal:

- `Y = A + B` when `Sel = 0`, utilizing the Adder (`cgm\_ADD8`).
- `Y = A \* B` when `Sel = 1`, utilizing the Multiplier (`cgm\_MULT8`).

The output `Y` (designated as `S`) is a 16-bit vector, accommodating the maximum multiplication result of $255 \times 255 = 65025$.

4.2 Schematic Capture

The ALU integrates the Adder (`cgm\_ADD8`) and Multiplier (`cgm\_MULT8`) with a 16-bit 2:1 multiplexer (`MUX16`) to select between the Adder's output (for addition) and the Multiplier's output (for multiplication) based on the `Sel` signal. The `MUX16` consists of 16 individual 2:1 multiplexers, each implemented using AND, OR, and NOT gates from the gpdk045 library. Each 2:1 multiplexer accepts two 1-bit inputs (from the Adder and Multiplier outputs) and the `Sel` signal, producing a single bit of the 16-bit output. Transistor sizing followed the same specifications as the Adder and Multiplier (120 nm NMOS, 240 nm PMOS), with adjustments in

the critical path to optimize the performance of the `MUX16`. The schematic of the 8-bit ALU is presented in Figure 13.

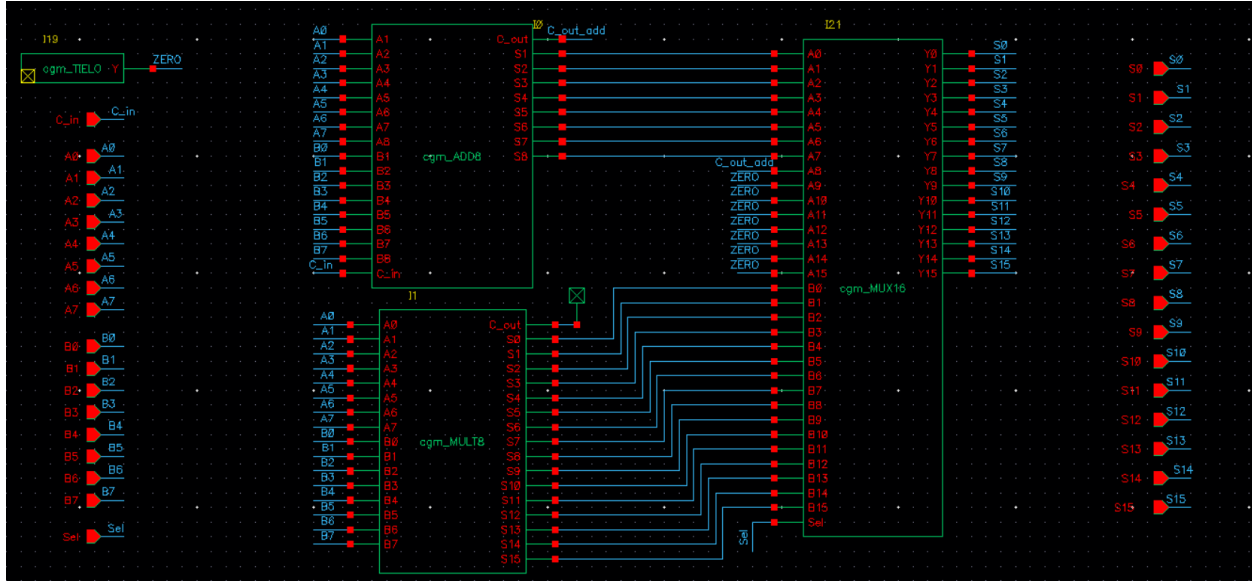


Figure 13: Schematic of the 8-bit ALU (`cgm\_ALU`) in Cadence Virtuoso, illustrating the integration of the Adder, Multiplier, and MUX16.

4.3 Layout Capture

Due to its complexity, the ALU layout was implemented across 1 row which takes the size of the maximum cell which is the MULT cell. VDD and VSS were positioned on Metal 1, adhering to the routing guidelines: 2 tracks at the top and bottom, 4 tracks between rows, and 30λ-wide VDD/VSS strapping. Power strapping was manually inserted, and auto-routing was restricted to signal nets using the Navigator tool to exclude VDD and VSS. Pin placement was optimized to facilitate future integration, such as for the Built-In Self-Test (BIST) implementation in Milestone 3. Routing was carefully managed to minimize parasitic effects. The layout of the 8-bit ALU is presented in Figure 14.

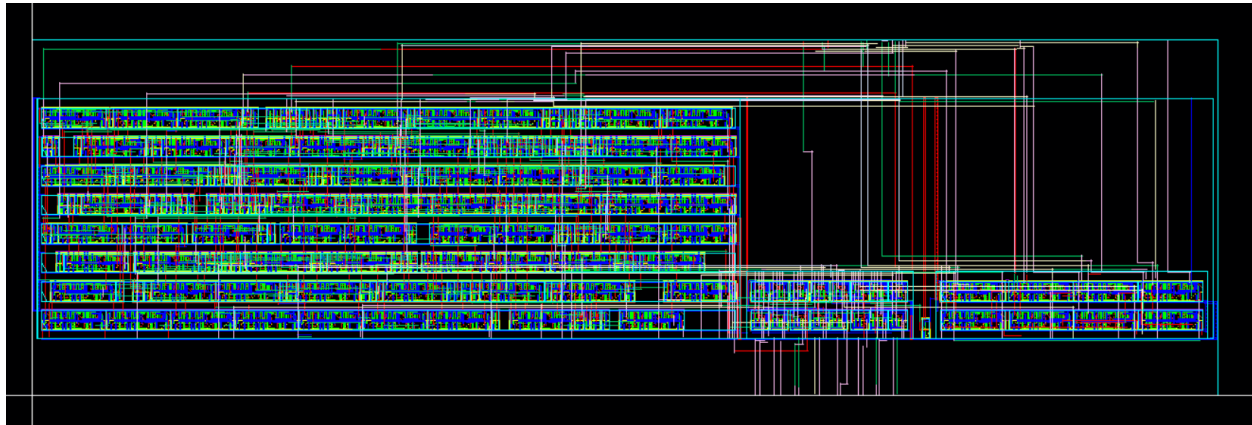


Figure 14: Layout of the 8-bit ALU (`cgm\_ALU`) in Cadence Virtuoso, depicting the 1-row structure with VDD/VSS on Metal 1.

The dimensions of the ALU layout, including width, height, and area, are to be determined from Cadence Virtuoso measurements, with adjustments for power strapping.

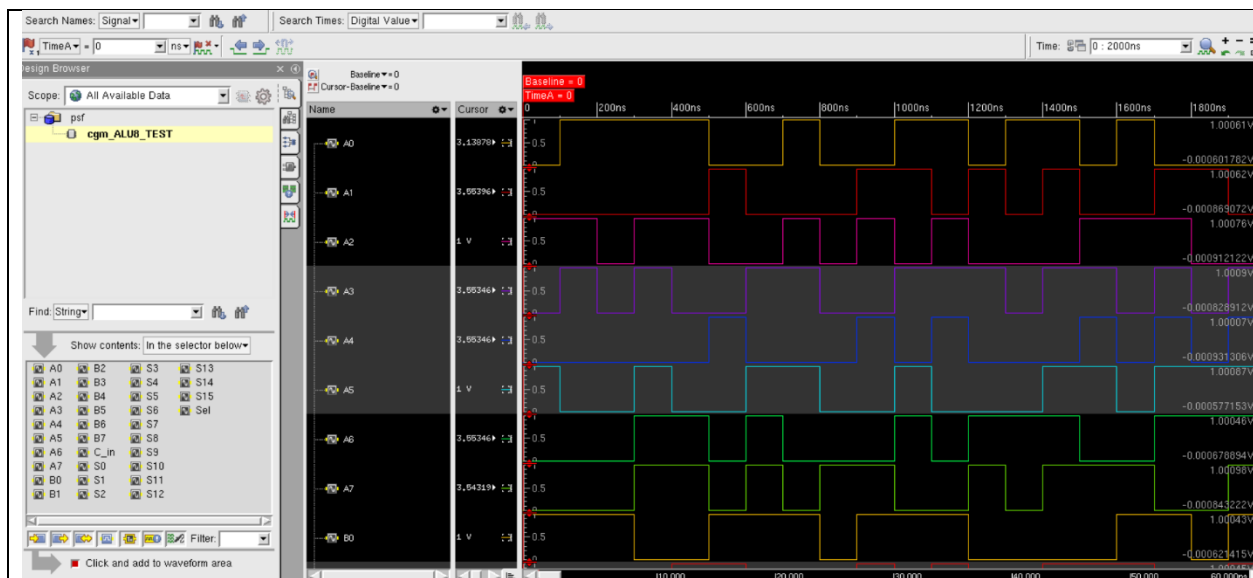
19

[illegible]

Figure 15: Width, height, and area used for the ALU8's layout.

4.5 Functional Simulation Waveform

Functional verification of the ALU was conducted using the testbench `cgm\_ALU8\_TESTBENCH`, encompassing 500 random test cases and edge cases: $255 + 255 + 1$ for addition (`Sel = 0`, `C\_in = 1`) and 255×255 for multiplication (`Sel = 1`). All test cases passed, confirming correct operation for both functions. Eight representative test cases were selected for documentation: six random cases (mixing addition and multiplication) and the two edge cases, demonstrating robustness across scenarios. The functional simulation waveform of the ALU, illustrating these eight test cases, is presented in Figure 16.



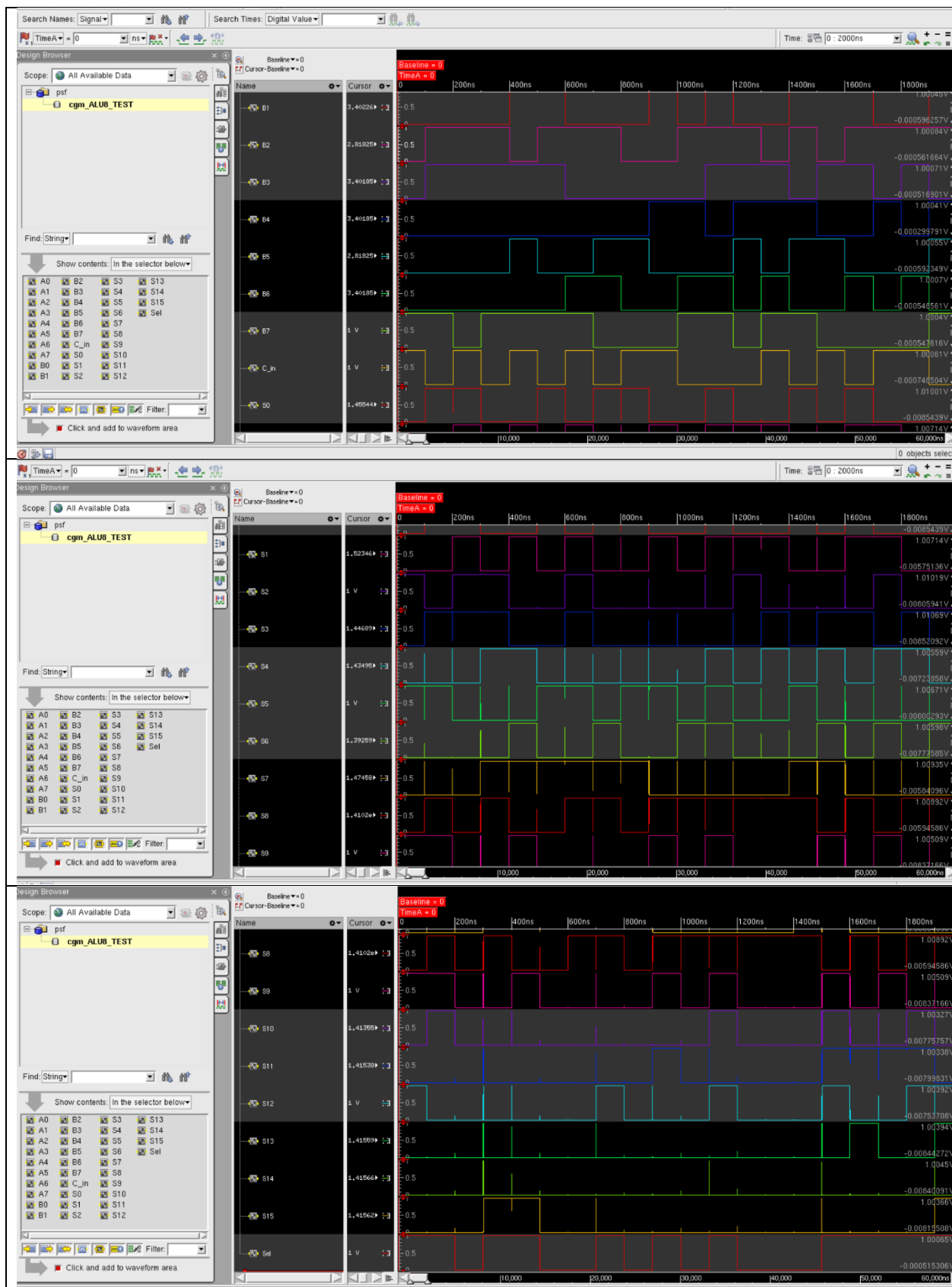
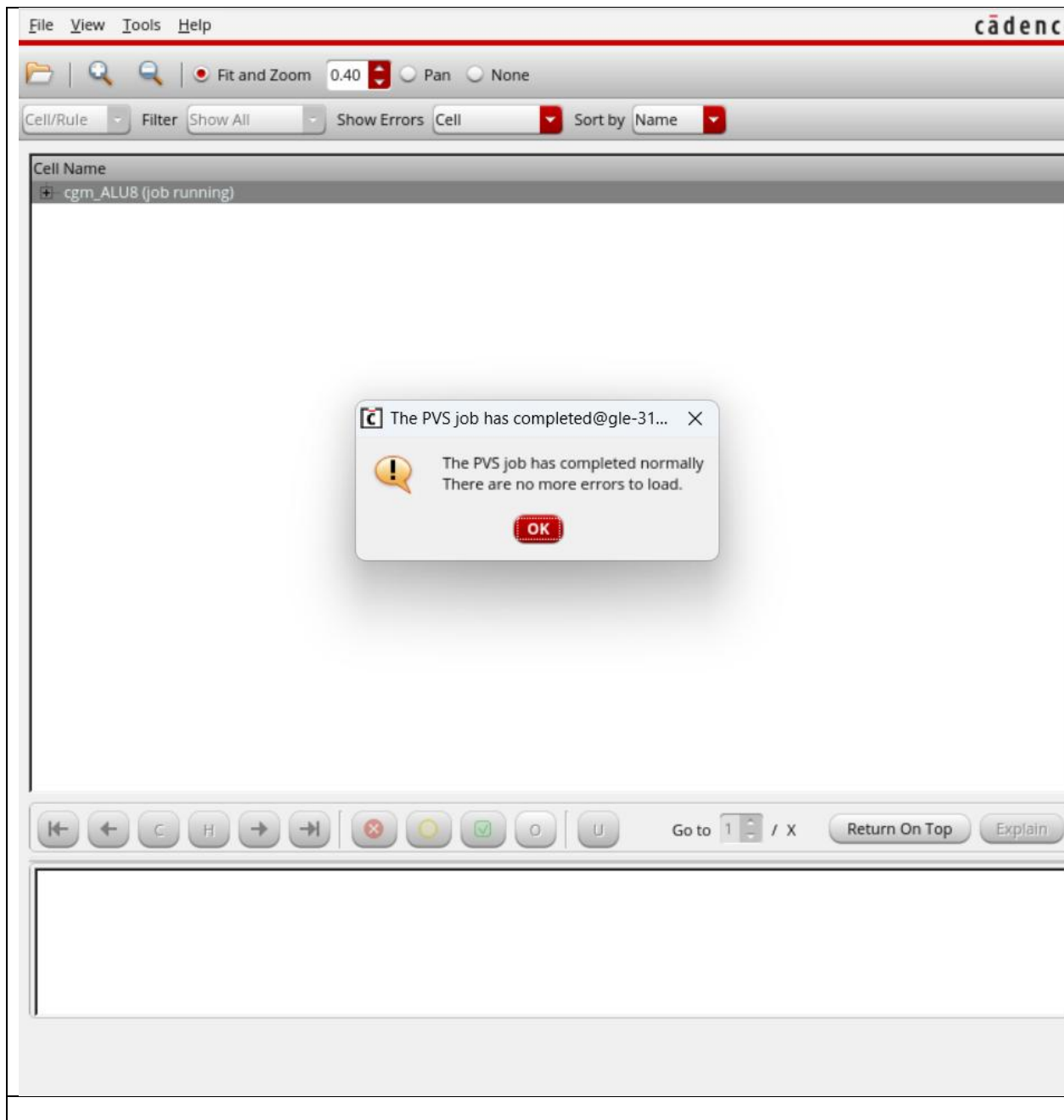
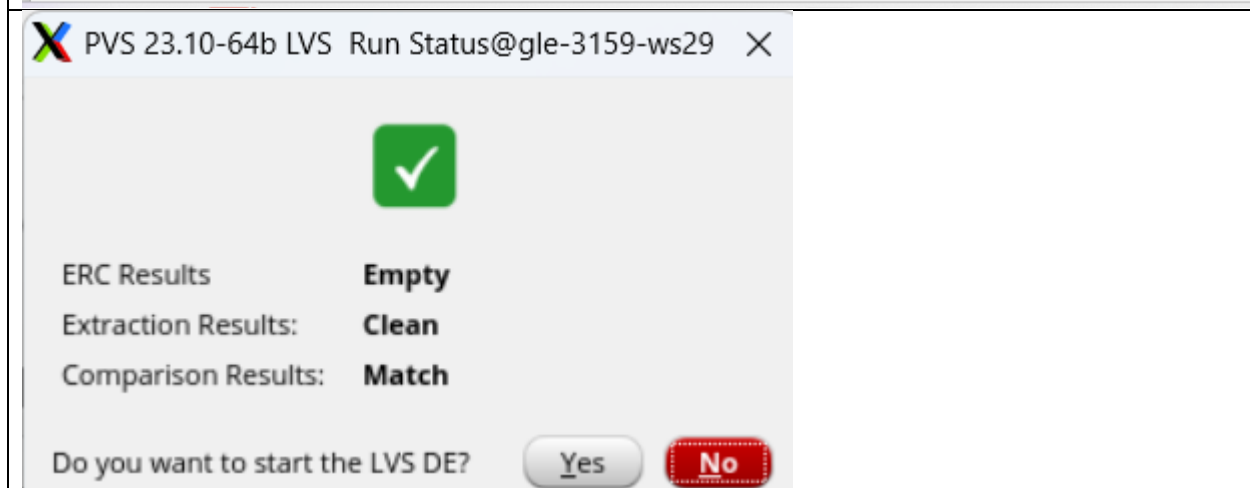
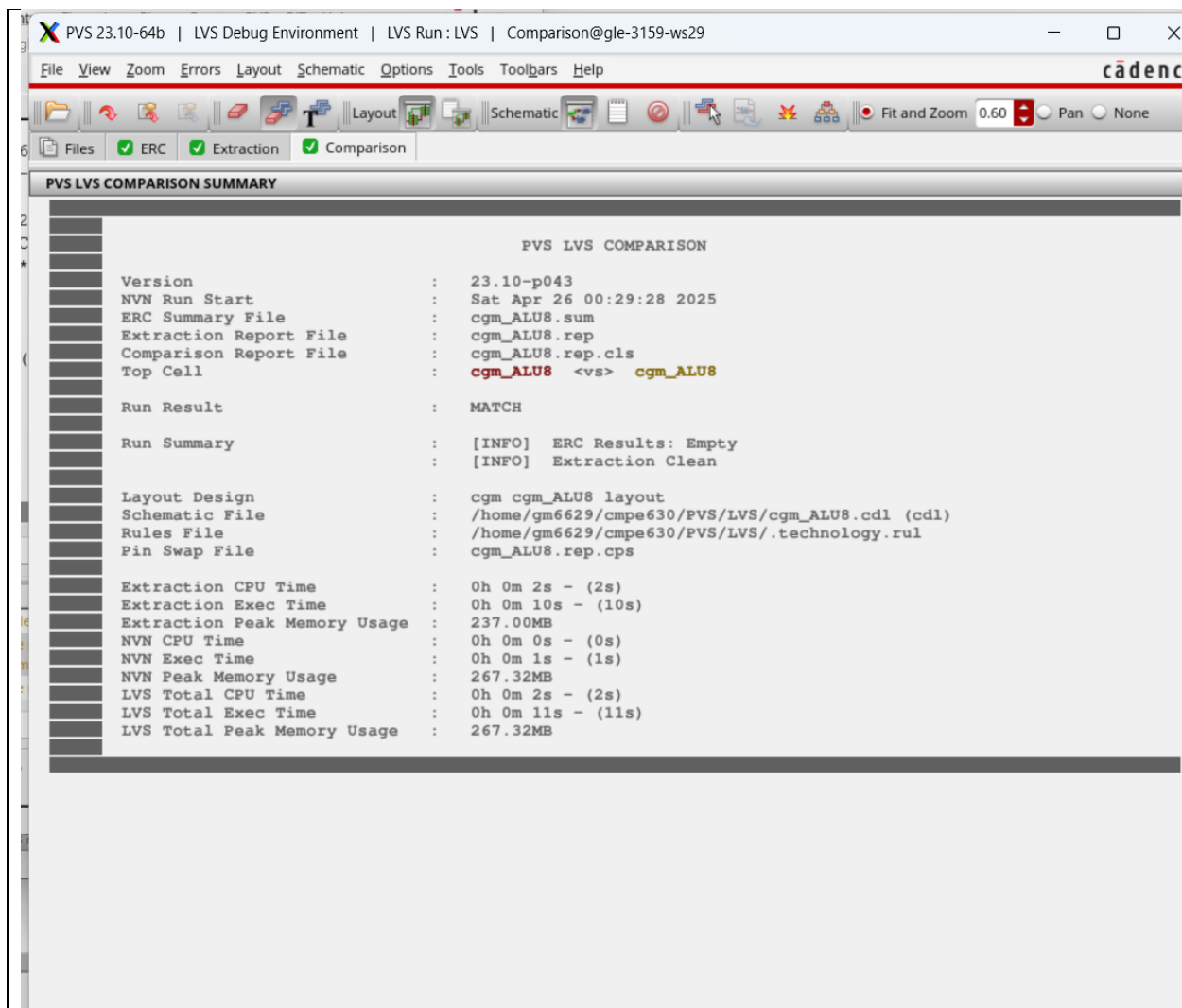


Figure 16: Functional simulation waveform of `cgm\_ALU` in SimVision, displaying eight representative test cases (six random cases with mixed addition and multiplication operations, and two edge cases: $255 + 255 + 1$ for addition, 255×255 for multiplication) from the 500 random test cases run by `cgm\_ALU8\_TESTBENCH`. The waveform includes inputs `A[7:0]`, `B[7:0]`, `Sel`, `C\_in`, and outputs `S[15:0]`.

4.6 Passing DRC, LVS, QRC Screenshots

The ALU layout underwent DRC, LVS, and QRC verification, addressing minor violations such as metal spacing and via enclosure issues. No “soft check” errors were detected, confirming proper VDD/VSS connectivity. The screenshots confirming passing DRC, LVS, and QRC checks for the ALU are presented in Figure 17.





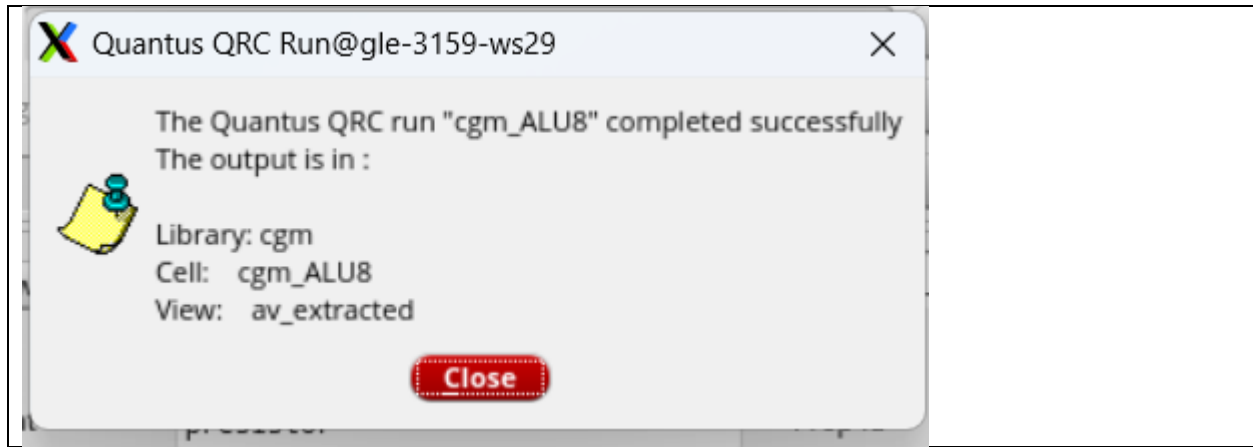


Figure 17: DRC, LVS, and QRC passing screenshots for `cgm\_ALU`.

4.7 Extracted Layout

The extracted layout includes parasitic effects, providing a realistic assessment of the ALU's performance with interconnect-related electrical characteristics. The extracted layout of the ALU is presented in Figure 18.

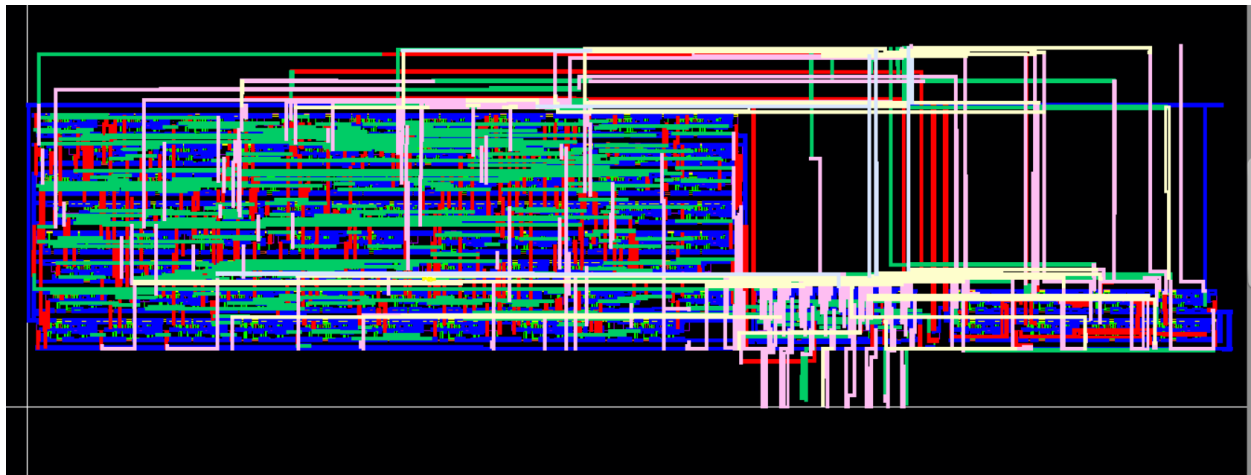


Figure 18: Extracted layout of `cgm\_ALU` in Cadence Virtuoso, showing the post-QRC view with parasitics.

4.8 Worst-Case Path Delay and Maximum Clock Frequency

A test circuit, `cgm\_ALU8\_TEST`, was developed to measure the ALU's worst-case delay, consisting of two ALUs connected in series to emulate realistic loading conditions (approximately 10-20 fF per bit in the extracted view, including parasitics). Two edge cases were evaluated: addition ($255 + 255 + 1$, `Sel = 0`, `C\_in = 1`) and multiplication (255×255 , `Sel = 1`).

The delay was measured in SimVision by calculating the time from the midpoint of the rise of input `A0` (least significant bit of `A`) to the midpoint of the rise of output `S15` (most significant bit of the result). The results are as follows:

Pre-Layout (Schematic):

- Worst-case delay: 417.4 ps (0.4174 ns), excluding layout effects.

Post-Layout (Extracted):

From the waveform:

- `A0` midpoint rise: 42.0279 μ s.
- `S15` midpoint rise: 42.001419 μ s.
- Delay: 42.0279 μ s - 42.001419 μ s = 26.5649 ns (rounded to 27 ns), reflecting the impact of parasitic effects such as interconnect capacitance.

The maximum clock frequency was calculated using the formula:

$$F_{max} = \frac{1}{Worst - Case Delay}$$

Pre-Layout:

- Delay: 400 ps = 0.4 ns = 0.4174×10^{-9} s.
- Frequency:

$$F_{max} = \frac{1}{(0.4174 \times 10^{-9} \text{ s})} = 2.4 \times 10^9 \text{ Hz}$$

Post-Layout:

- Delay: 26 ns = 27×10^{-9} s.
- Frequency:

$$F_{max(post-layout)} = \frac{1}{(27 \times 10^{-9} \text{ s})} = 0.03846 \times 10^9 \text{ Hz} = 37 \text{ MHz}$$

The waveform plot illustrating this delay measurement for the ALU test circuit is presented in Figure 19.

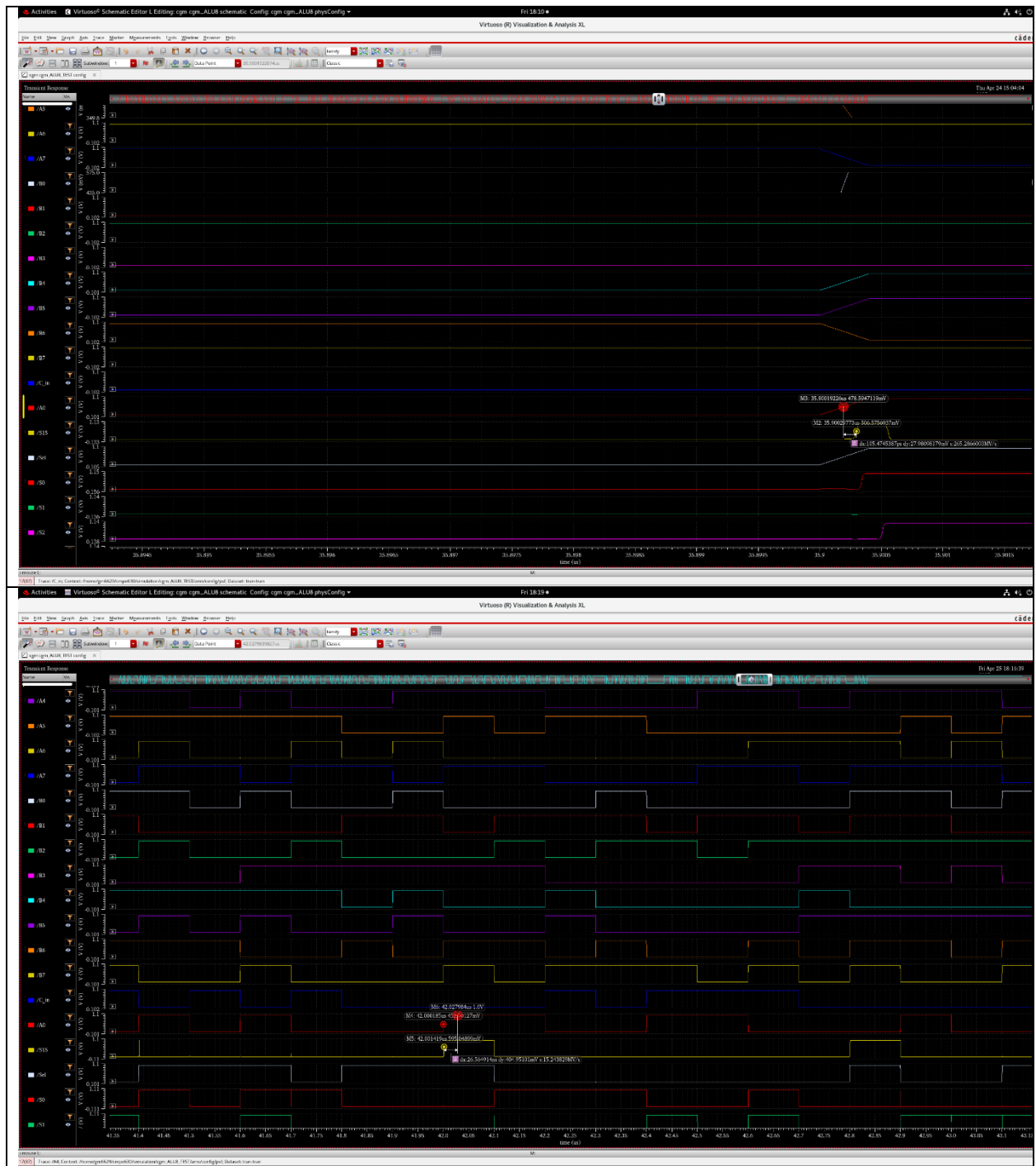


Figure 19 Waveform plot in SimVision showing post-layout delay measurement for `cgm\_ALU8\_TEST`.

Conclusion

The 8-bit ALU, comprising the Adder (`cgm\_ADD8`), Multiplier (`cgm\_MULT8`), and a 16-bit 2:1 multiplexer (`MUX16`), was successfully designed, implemented, and verified in Cadence Virtuoso using the 45nm gpdK045 process for Milestone 2 of the CMPE 630 final project. Functional simulations confirmed correct operation across all modules, with waveforms documenting eight passing test cases per module. All layouts passed DRC, LVS, and QRC verification. The ALU exhibited a pre-layout worst-case delay of 417 ps, corresponding to a maximum clock frequency of 2.4 GHz, while the post-layout delay increased to 27 ns due to parasitic effects, reducing the frequency to 37 MHz. Challenges in layout implementation, particularly in managing multi-row configurations, were addressed through transistor folding and optimized routing strategies.

Appendix: Relevant Codes

6.1 Adder Testbench

The following Verilog testbench was employed to verify the Adder. It applies 500 random test cases to inputs `A`, `B`, and `C\_in`, validates the output `sum` (bits `S1` to `S8`) and `C\_out` against the expected result `A + B + C\_in`, and includes an edge case of 255 + 255 + 1:

```
//VERILOG HDL FOR "CGM", "CGM_ADD8_TESTBENCH" "FUNCTIONAL"
```

```
MODULE CGM_ADD8_TESTBENCH ( A1, A2, A3, A4, A5, A6, A7, A8, B1, B2, B3, B4, B5, B6, B7, B8, S1, S2, S3, S4, S5, S6, S7, S8, C_OUT, C_IN);
INPUT S1, S2, S3, S4, S5, S6, S7, S8, C_OUT;
OUTPUT A1, A2, A3, A4, A5, A6, A7, A8, B1, B2, B3, B4, B5, B6, B7, B8, C_IN;

WIRE A1, A2, A3, A4, A5, A6, A7, A8;
WIRE B1, B2, B3, B4, B5, B6, B7, B8;
REG [8:1]A = 0;
REG [8:1]B = 0;
REG C_IN;
WIRE [8:1] SUM = {S8, S7, S6, S5, S4, S3, S2, S1};
INTEGER I;
INTEGER PASSED;
ASSIGN {A8, A7, A6, A5, A4, A3, A2, A1} = A;
ASSIGN {B8, B7, B6, B5, B4, B3, B2, B1} = B;

//ASSIGN C_OUT = R9;

INITIAL
BEGIN
    PASSED = 0;
    FOR (I = 0; I < 500; I = I + 1) BEGIN
        A = $RANDOM();
        B = $RANDOM();
        C_IN = $RANDOM() %2;

        $DISPLAY ("ROUND %D A = %B, B = %B, C_IN = %D" , I, A, B, C_IN);

        #10;

        $DISPLAY(" SUM = %B, C_OUT = %B" , SUM, C_OUT);
```

```

        #10;

        IF((SUM + (C_OUT << 8)) != A + B + C_IN) BEGIN
            $DISPLAY(" IT FAILED");
            PASSED = PASSED - 1;
        END

        PASSED = PASSED + 1;
    END

    $DISPLAY("PASSED TESTS = %D" , PASSED);

    #10;

    A = 255;
    B = 255;
    C_IN = 1;

    $DISPLAY ("ROUND %D A = %B, B = %B, C_IN = %D" , I, A, B, C_IN);

    #10;

    $DISPLAY(" SUM = %B, C_OUT = %B" , SUM, C_OUT);

    #10;

    IF((SUM + (C_OUT << 8)) != A + B + C_IN) BEGIN
        $DISPLAY(" IT FAILED");
    END

    END

ENDMODULE

```

6.2 Multiplier Testbench

The following Verilog testbench was employed to verify the Multiplier. It applies 500 random test cases to inputs `A` and `B`, validates the output against `A \* B`, and includes an edge case of 255×255 :

```

//Verilog HDL for "cgm", "cgm_MULT8_TESTBENCH" "functional"

module cgm_MULT8_TESTBENCH (
    A0, A1, A2, A3, A4, A5, A6, A7, // Inputs A[7:0]
    B0, B1, B2, B3, B4, B5, B6, B7, // Inputs B[7:0]
    S0, S1, S2, S3, S4, S5, S6, S7, // Outputs S[15:0]
    S8, S9, S10, S11, S12, S13, S14, S15,
    C_out // Carry-out (updated to C_out)
);

output A0, A1, A2, A3, A4, A5, A6, A7, B0, B1, B2, B3, B4, B5, B6, B7;
input S0, S1, S2, S3, S4, S5, S6, S7, S8, S9, S10, S11, S12, S13, S14, S15, C_out;

// Wires for individual bits of A and B
wire A0, A1, A2, A3, A4, A5, A6, A7;
wire B0, B1, B2, B3, B4, B5, B6, B7;

// Registers to drive inputs
reg [7:0] A = 0;
reg [7:0] B = 0;

```

```

// Combine the output bits into a 16-bit result
wire [15:0] result = {S15, S14, S13, S12, S11, S10, S9, S8, S7, S6, S5, S4, S3, S2, S1, S0};

// Assign the individual bits of A and B
assign {A7, A6, A5, A4, A3, A2, A1, A0} = A;
assign {B7, B6, B5, B4, B3, B2, B1, B0} = B;

// Variables for test loop and pass/fail tracking
integer i;
integer passed;

initial begin
    passed = 0;

    // Run 500 random test cases
    for (i = 0; i < 500; i = i + 1) begin
        // Generate random 8-bit inputs
        A = $random() % 256; // 0 to 255
        B = $random() % 256; // 0 to 255

        $display("Round %d A = %b, B = %b", i, A, B);

        #30; // Wait for the multiplier to compute the result

        $display("Result = %b, C_out = %b", result, C_out);

        #30; // Additional delay to observe the result

        // Compute expected result (A * B)
        // Since A and B are 8-bit, A * B is 16-bit (max 255 * 255 = 65025)
        // Check if S[15:0] matches the expected product directly
        if (result !== (A * B)) begin
            $display("It failed");
            passed = passed - 1;
        end

        passed = passed + 1;
    end

    $display("Passed tests = %d", passed);

    #30;

    // Edge case: A = 255, B = 255 (max inputs)
    A = 255; // 8'hFF
    B = 255; // 8'hFF

    $display("Edge Case: A = %b, B = %b", A, B);

    #30;

    $display("Result = %b, C_out = %b", result, C_out);

    #30;

    if (result !== (A * B)) begin
        $display("It failed");
    end
end

endmodule

```

6.3 ALU Testbench

The following Verilog testbench was employed to verify the ALU. It applies 500 random test cases to inputs `A`, `B`, `Sel`, and `C\_in`, validates the output against expected results for addition or multiplication, and includes edge cases ($255 + 255 + 1$, 255×255):

```
//Verilog HDL for "cgm", "cgm_ALU8_TESTBENCH" "function"
```

```
module cgm_ALU8_TESTBENCH (
    A0, A1, A2, A3, A4, A5, A6, A7, // Inputs A[7:0]
    B0, B1, B2, B3, B4, B5, B6, B7, // Inputs B[7:0]
    Sel, // Select signal
    C_in, // Carry-in for ADD8
    S0, S1, S2, S3, S4, S5, S6, S7, // Outputs S[15:0]
    S8, S9, S10, S11, S12, S13, S14, S15
);

// Declare the ports with correct directions
output A0, A1, A2, A3, A4, A5, A6, A7;
output B0, B1, B2, B3, B4, B5, B6, B7;
output Sel;
output C_in;
input S0, S1, S2, S3, S4, S5, S6, S7, S8, S9, S10, S11, S12, S13, S14, S15;

// Wires for individual bits of A and B
wire A0, A1, A2, A3, A4, A5, A6, A7;
wire B0, B1, B2, B3, B4, B5, B6, B7;

// Registers to drive inputs
reg [7:0] A = 0; // 8-bit input A
reg [7:0] B = 0; // 8-bit input B
reg Sel; // Select signal
reg C_in; // Carry-in

// Assign the individual bits of A and B to the output ports
assign {A7, A6, A5, A4, A3, A2, A1, A0} = A;
assign {B7, B6, B5, B4, B3, B2, B1, B0} = B;

// Combine the output bits into a 16-bit sum for easier comparison
wire [15:0] sum = {S15, S14, S13, S12, S11, S10, S9, S8,
    S7, S6, S5, S4, S3, S2, S1, S0};

// Variables for test loop and pass/fail tracking
integer i;
integer passed;

// Variables for expected addition result
wire [8:0] expected_add_result = A + B + C_in; // 9-bit result: [carry][sum[7:0]]
wire expected_c_out = expected_add_result[8]; // Expected carry-out (should match S[8])
wire [7:0] expected_sum = expected_add_result[7:0]; // No reversal, matches S1-S8 in sum[7:0]

initial begin
    passed = 0;

    // Run 500 random test cases
    for (i = 0; i < 500; i = i + 1) begin
        // Generate random 8-bit inputs, Sel, and C_in
        A = $random() % 256; // 0 to 255
        B = $random() % 256; // 0 to 255
```

```

Sel = $random() % 2; // 0 or 1
C_in = $random() % 2; // 0 or 1

$display("Round %d: A = %b, B = %b, Sel = %b, C_in = %b", i, A, B, Sel, C_in);

#50; // Wait for the ALU to compute the result

$display("Sum (S15-S0) = %b", sum);

#50; // Additional delay to observe the result

// Check the result based on Sel
if (Sel == 0) begin
    // Addition: S[15:0] = {7'b0, C_out_add, S1-S8}
    // Check carry-out (S[8] should match expected carry)
    if (sum[8] !== expected_c_out) begin
        $display("Addition carry failed: Expected %b, Got %b", expected_c_out, sum[8]);
        passed = passed - 1;
    end
    // Check S1-S8 (S[0:7] should match expected sum, no reversal)
    if (sum[7:0] !== expected_sum) begin
        $display("Addition sum failed: Expected %b, Got %b", expected_sum, sum[7:0]);
        passed = passed - 1;
    end
    // Check S[15:9] are zero (padded by Tielo)
    if (sum[15:9] !== 7'b0) begin
        $display("Addition upper padding failed: Expected 7'b0, Got %b", sum[15:9]);
        passed = passed - 1;
    end
end else begin
    // Multiplication: S[15:0] = A * B
    // A * B is 16-bit (max 255 * 255 = 65025)
    if (sum !== (A * B)) begin
        $display("Multiplication failed: Expected %b, Got %b", (A * B), sum);
        passed = passed - 1;
    end
end

passed = passed + 1;
end

$display("Passed tests = %d", passed);

#50;

// Edge Case 1: A = 255, B = 255, Sel = 0, C_in = 1 (Addition with overflow)
A = 255; // 8'hFF
B = 255; // 8'hFF
Sel = 0; // Addition
C_in = 1; // Carry-in

$display("Edge Case 1: A = %b, B = %b, Sel = %b, C_in = %b", A, B, Sel, C_in);

#50;

$display("Sum (S15-S0) = %b", sum);

#50;

if (sum[8] !== expected_c_out) begin
    $display("Edge Case 1 (Addition carry) failed: Expected %b, Got %b", expected_c_out, sum[8]);
end

```

```

if (sum[7:0] !== expected_sum) begin
    $display("Edge Case 1 (Addition sum) failed: Expected %b, Got %b", expected_sum, sum[7:0]);
end
if (sum[15:9] !== 7'b0) begin
    $display("Edge Case 1 (Addition upper padding) failed: Expected 7'b0, Got %b", sum[15:9]);
end

#50;

// Edge Case 2: A = 255, B = 255, Sel = 1, C_in = 0 (Multiplication with max result)
A = 255; // 8'hFF
B = 255; // 8'hFF
Sel = 1; // Multiplication
C_in = 0; // C_in is ignored for multiplication

$display("Edge Case 2: A = %b, B = %b, Sel = %b, C_in = %b", A, B, Sel, C_in);

#50;

$display("Sum (S15-S0) = %b", sum);

#50;

if (sum !== (A * B)) begin
    $display("Edge Case 2 (Multiplication) failed: Expected %b, Got %b", (A * B), sum);
end
end

endmodule

```